02 DATA TYPES AND VARIABLES — C++ Programs Reference PDF

Annotated C++ Source Programs — 02 DATA TYPES AND VARIABLES

Program: `01_fundamental_types_and_sizes.cpp`

C++ SOURCE FILE • 01_fundamental_types_and_sizes.cpp

```cpp
// Program Title: Fundamental Data Types, Initialization, and Value Ranges
// Topic: Data Types and Variables
// Purpose: Demonstrate initializing fundamental C++ types and printing their values.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_fundamental_types_and_representations.md
// Program Classification: Category A – Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int count{42};
    double ratio{3.5};
    char grade{'A'};
    bool passed{true};

    cout << "count: " << count << ", ratio: " << ratio
         << ", grade: " << grade << ", passed: " << passed << endl;
    return 0;
}

/*
Expected Deterministic Output:
count: 42, ratio: 3.5, grade: A, passed: 1
*/
```

Program: `02_const_and_constexpr.cpp`

C++ SOURCE FILE • 02_const_and_constexpr.cpp

```cpp
// Program Title: Compile-Time Constants Using constexpr and const
// Topic: Data Types and Variables — const and constexpr
// Purpose: Use a constexpr constant as a compile-time array size bound.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_const_constexpr_and_auto.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    constexpr int SIZE = 4;
    int squares[SIZE] = {1, 4, 9, 16};

    for (int i = 0; i < SIZE; i++) {
        cout << squares[i] << " ";
    }
    cout << endl;
    return 0;
}

/*
Expected Deterministic Output:
1 4 9 16
*/
```

Program: 03_narrowing_conversion_compile_error.cpp

C++ SOURCE FILE • 03_narrowing_conversion_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Narrowing Conversion Rejected by Brace Initialization
// Topic: Data Types and Variables — List Initialization Rules
// Purpose: Show how brace initialization {3.14} catches floating-to-int narrowing at compile
time.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/02_variable_initialization_and_narrowing.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

int main() {
    int x{3.14}; // COMPILE ERROR: narrowing conversion of '3.14' from 'double' to
'int'
    cout << x << endl;
    return 0;
}

/*
Why Compilation Fails:
ISO C++ [dcl.init.list] forbids narrowing conversions inside brace-init-lists '{}'.
Use a matching type 'double x{3.14};' or an explicit cast 'int x{static_cast<int>(3.14)};'.
*/
```